OpenAgenet/OAN

Open Governance Infrastructure for Trusted Agent Interconnection

Identity, Governance, Registration, Distribution, and Trusted Discovery

OAN slogan:

OAN helps the Agent Internet move from jungle rules to rule-based trust.

Jinliang Xu

China Academy of Information and Communications Technology (CAICT)

Agenda

If you remember only one sentence

OAN is the trust infrastructure layer for agent resources: think of it as a governed resource registry, a verifiable identity layer, and a trusted discovery network in one stack.

1. What problem OAN solves for developers and operators
2. What OAN is, in plain language
3. Why OAN is not just "another protocol"
4. How governance, Root, Registrar, and Discovery work together
5. How `did:oan` supports portable agent resources
6. How OAN complements MCP, A2A, ANP, and APIs
7. What engineering evidence already exists
8. What performance looks like today
9. How to try, join, and build on OAN

Three takeaways

OAN adds trust before invocation, governance before infrastructure authority, and evidence before discovery results.

Why Multi-Agent Collaboration Becomes Inevitable

What the original argument actually says

Multi-agent collaboration is not presented here as a fashionable architecture choice. The underlying claim is stronger: once real-world actors, resources, and responsibilities are agentized, collaboration becomes structurally unavoidable at both the ecosystem level and the engineering level.

1. Macro perspective

- Knowledge, tools, data sources, and responsibilities are already distributed across people, organizations, websites, systems, and domains.
- When those actors evolve into software agents, the distributed structure does not disappear; it becomes an inter-agent structure.
- In that world, collaboration stops being an exception and becomes the default operating form of the ecosystem.

2. Engineering perspective

- A single super-agent eventually hits coordination cost, scaling limits, and trust-domain boundaries.
- Decomposing work across multiple agents improves modularity, parallelism, fault isolation, and organizational fit.
- But once collaboration crosses teams or domains, it can no longer rely on private trust assumptions or ad hoc endpoint exchange.

Why this leads to OAN

If multi-agent collaboration is the natural endpoint of both real-world distribution and engineering decomposition, then open identity, governance, registration, distribution, and trusted discovery become infrastructure requirements, not optional add-ons.

OAN in Plain Language

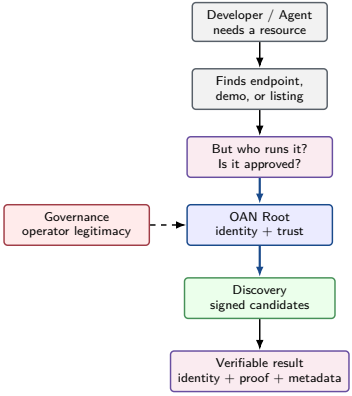
One-sentence explanation

OAN is like a **trusted directory + identity verification center** for agent resources:

- it gives resources a verifiable identity;
- it lets infrastructure operators work under governance;
- it lets users find candidates with signed evidence before they connect.

Why that matters

- Developers need to know **who stands behind an endpoint**.
- Operators need **governed legitimacy**, not just uptime.
- Discovery is not enough if the result cannot be **verified before use**.



Short version

OAN is the missing trust layer between “I found an agent resource” and “I can safely use it”.

The Real Developer Pain

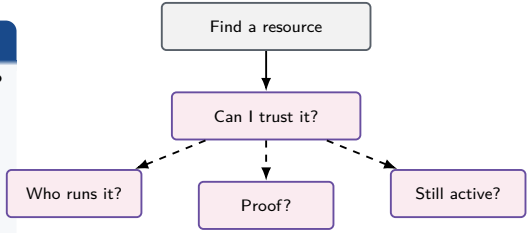
The Pain Point

Before introducing architecture, it helps to anchor the problem in an everyday developer situation. This page shows why finding an agent resource is easy, but deciding whether it is trustworthy is still too often a guess.

Typical scenario

A developer wants to call someone else's Agent service, Skill, MCP server, or API:

1. They can find endpoints or demos.
2. But they cannot easily verify whether the resource is authentic and governance-approved.
3. Every connection becomes a trust gamble: **real or fake, active or revoked, safe or risky?**

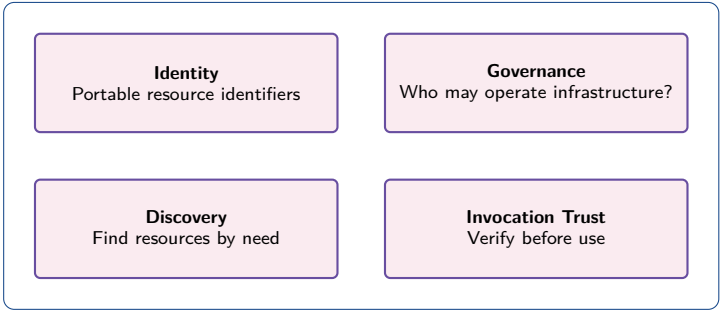


Why OAN exists

OAN is built to turn that trust gamble into a governed, verifiable, evidence-carrying workflow.

Source: White paper: Background and Why OAN; product model: Goal/Core Positioning

What the Ecosystem Is Missing Today



- Existing protocols connect endpoints, but they do not answer: **who is legitimate, what is current, and what is safe to trust**.
- OAN targets the missing layer between connectivity and trust.

Why this becomes painful at scale

At ecosystem scale, treating "discoverable" as if it already meant "trustworthy" becomes a systemic risk.

Source: White paper: Identity/Governance/Discovery/Invocation gaps; identity trust layering

OAN's Core Answer

The Core Move

Once the trust gap is clear, OAN's answer can be stated very simply. It combines open governance, portable resource identity, and trusted discovery into one coherent infrastructure layer.

Positioning

OAN is not just a registry. It is a protocol-neutral trust infrastructure layer for agent resources: governance-backed infrastructure, portable resource identity, and trusted discovery with evidence.

DID = decentralized identifier; VC = verifiable credential.

Source: White paper: Vision and Scope; yellow paper: Protocol Stack and Trust Layers

Mini Legend for Non-Specialists

Quick Vocabulary

The next few pages use terms from decentralized identity, governance, and agent interoperability. This quick legend lowers the barrier so readers can follow the diagrams without already knowing the full vocabulary.

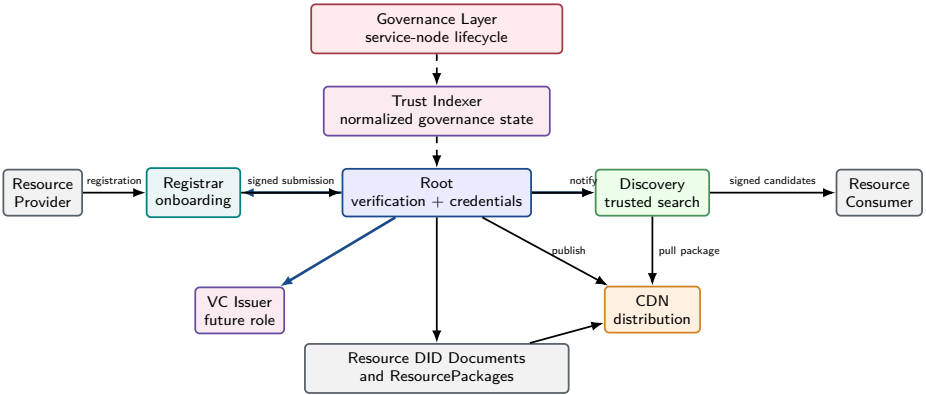
Term	Plain-language meaning
DID	Portable digital identity card for a resource.
VC	Signed permit or attestation from a trusted party.
MCP	Protocol for models and tools to exchange context and tools.
A2A	Protocol for agents to talk to each other.
ANP	Adjacent agent-network route OAN can complement.
Root	OAN trust anchor that verifies, accepts, and authorizes.

Why this matters

Readers should be able to understand the diagrams even if they are not already experts in decentralized identity or protocol standards.

Source: White paper terminology; DID method spec; security model

OAN Architecture



Governance decides who may operate nodes; Root decides what may be accepted and published; Discovery decides what may be found with signed evidence.

Dashed arrows = observation/synchronization; solid arrows = trust-carrying operational flow.

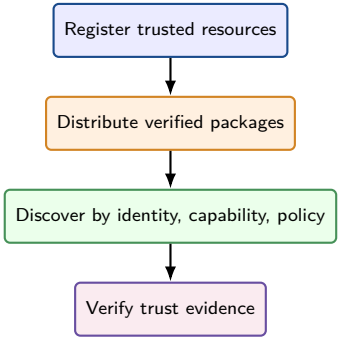
Source: System design: Roles/Trust Flow; on-chain bulletin; trust indexer design

What OAN Is

System Boundary

Now that the key terms and the top-level architecture are on the table, this page makes the system boundary concrete. It summarizes OAN as a full trust pipeline rather than a mere identifier format or another registry interface.

- A trust-governed resource identity and discovery infrastructure.
- A registration, publication, distribution, discovery, and verification path for AI-agent-related resources.
- A common layer above heterogeneous interaction protocols.



Source: White paper: What OAN Is; product model: Core Positioning

Where OAN Fits

Stack Position

Once OAN is defined positively, the next step is to place it relative to familiar systems. This page clarifies where OAN sits in the stack so readers do not mistake it for a communication protocol or an ordinary marketplace.

Positioning	Meaning
Works above MCP/A2A/ANP/APIs	OAN does not replace communication protocols; it gives them a governed identity and trust layer.
Focused on trusted registration and discovery	OAN is not a generic app marketplace; it is the infrastructure that decides what is publishable and discoverable.
Designed for evidence before use	OAN improves trust in identity, lifecycle, and discovery, before endpoint invocation happens.
Open and inspectable by design	Governance state, resource identity, and discovery evidence are meant to be reviewable, not platform-private.

Takeaway

OAN is the trust layer under the agent ecosystem, not another isolated upper-layer protocol.

Source: White paper: Core positioning; on-chain bulletin Non-goals; product model boundaries

Why This Architecture Is Valuable

Why It Matters

After positioning OAN in the stack, it helps to translate that placement into concrete benefits. This page connects the architectural choices to both technical resilience and developer-facing value.

Technical Advantages

- Role separation means one compromise or one operator does not define the whole network.
- Governance visibility makes lifecycle and revocation observable instead of implicit.
- Root authorization makes infrastructure trust an explicit decision, not just a URL claim.
- DID-based identity keeps resources portable across changing protocols and endpoints.

Developer / Ecosystem Benefits

- Developers spend less time guessing which resources are legitimate.
- Operators can join as governed infrastructure instead of informal directories.
- Discovery can evolve into domain-specific trusted ecosystems.
- Open specifications make OAN easier to review, adopt, and extend.

Human version

OAN helps developers avoid connecting to the wrong thing, and helps ecosystems avoid scaling on top

Where OAN Is Sharply Different

Dimension	OAN	BUPT-AIP	CAS-AIP	ANP	AGNTCY ADS	NANDA
Core positioning	Trust governance infra	Agent protocol family	Scientific agent runtime	Decentralized agent protocol	Distributed agent directory	Agent indexing and discovery
Primary object	Discoverable resource	Agent	Agent / ToolBox	Agent / DID endpoint	Agent descriptor / artifact	Agent facts / index
Governance model	On-chain/committee + Root	Layered multi-centers	Centralized / gateway	Community governance	LF / TSC / workgroups	Research alliance
Trust model	Layered trust + Root proof	Review + certs	mTLS / session trust	DID / E2EE / DNS	Artifact signing + DHT	Verifiable facts
Resource model	Agent service, skill, MCP, tool/API	Agent-centered ACS	Agent and ToolBox	Agent-centered	Agent / OASF centered	AgentFacts centered
Discovery	Authorized; GRAIL option	Embedding + filters	Secondary concern	Network / community based	Kademlia DHT directory	Adaptive resolver
Semantic discovery	GRAIL option	Embedding + filters	Secondary concern	Light / protocol side	Directory-first	Adaptive discovery
Identity	`did:oan`	AIC / OID style	Local IDs / AgentInfo	DID / did:wba	Identity + artifact IDs	AgentFacts IDs
Runtime protocol	Neutral; MCP/A2A/ANP/AIP adapters	AIP Direct RPC / Group	gRPC	ANP protocol stack	SLIM / ACP integration	Secondary concern
Implementation language	Rust core	Python core	Python core	Python core	Rust core/ Go components	Python core
Standardization	Paper + IETF drafts + CCSA work	Domestic standard alignment	Paper + prototype	Community specs + drafts	IETF + LF process	Academic papers
Ecosystem base	OpenAgenet + pilots	Domestic pilot ecosystem	Research / lab ecosystem	Open community + SDKs	Linux Foundation ecosystem	Academic network
Main strength	Pre-connection trust	Domestic pilot alignment	Scientific gRPC runtime	DID-based decentralization	Ecosystem + LF backing	Academic narrative

Source: Comparative analysis: OAN vs. BUPT-AIP, CAS-AIP, ANP, AGNTCY, NANDA

Where OAN Is Sharply Different

What Makes It Different

The useful comparison is not whether OAN can also move messages, but which trust responsibilities it adds that adjacent systems usually leave vague or externalized. This table is meant to make that contrast immediately visible.

Capability	Adjacent systems	OAN	Why it matters
Connectivity / invocation	✓	✓	OAN complements protocol connectivity.
Governed node authorization	×	✓	OAN decides who may operate infrastructure.
Identity before exposure	partial	✓	Resources carry governed identity before discovery.
Signed discovery evidence	rare	✓	Discovery results come with verifiable proof.
Cross-domain trust	limited	✓	OAN is built for multi-operator trust.

Bottom line

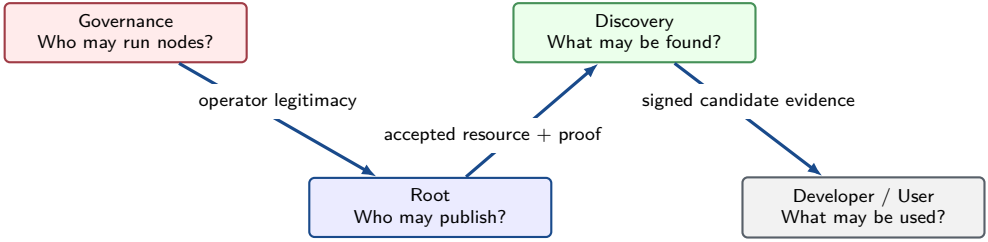
OAN is not valuable because it can also send messages. It is valuable because it adds the trust and governance features that adjacent systems usually leave out.

Source: White paper: Relationship with MCP/A2A/ANP; yellow paper: compatibility boundaries

The Trust Chain, Simplified

The Shortest Trust Story

The earlier comparison tables explain what OAN adds; this page compresses that answer into one simple trust path. It is the shortest route from governance legitimacy to a verifiable resource a user can safely choose.



What to remember

Governance establishes who may operate. Root establishes what is accepted. Discovery returns what is visible. Users verify before invocation.

Translation to plain language: first approve the operator, then approve the resource, then expose only verifiable results.
DID = decentralized identifier; VC = verifiable credential; MCP = Model Context Protocol.

Source: On-chain bulletin boundary; trust indexer design; resource-first model

OAN's Differentiated Value

Value Summary

This page turns the trust chain into a more explicit value statement. It summarizes both the extra responsibilities OAN takes on and the ecosystem-level benefits those responsibilities create.

What OAN Adds

- Governance before infrastructure authorization.
- Resource identity before directory exposure.
- Root acceptance before publication.
- Evidence before Discovery indexing.
- Revocation visibility before stale trust can spread.
- Portable trust proofs before cross-platform reuse.
- Verification before invocation.

Why It Matters

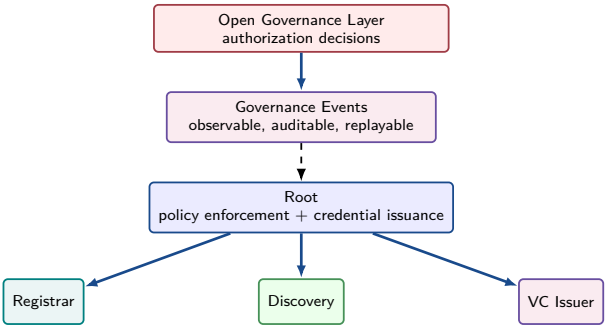
- Supports multi-operator and cross-domain trust.
- Covers Agent services, skills, MCP servers, and tool/API resources.
- Lets native protocols continue after trust is established.
- Keeps Discovery bounded by authorized domains and verifiable evidence.
- Provides a standards-facing trust model instead of a closed platform.

Positioning statement

OAN is a protocol-neutral trust and discovery infrastructure layer: it complements communication frameworks by governing resource identity, lifecycle, publication, and pre-connection verification.

Source: Agent interoperability comparison analysis: OAN core advantages; resource-first model; layered trust model

Who Controls the Root?



Who Actually Decides

- Governance decides which infrastructure identities are legitimate.
- Those decisions become replayable events rather than private operator judgment.
- Root consumes that governance state and turns it into operational trust decisions.
- Registrar, Discovery, and future issuers do not define trust on their own; they inherit it through Root.
- That makes Root powerful, but not sovereign: its authority is constrained by open governance evidence.

Message

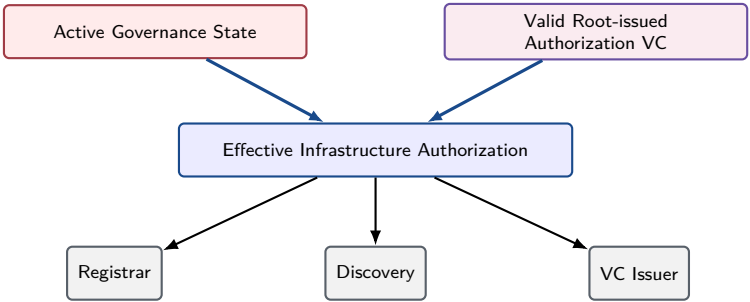
Root is not a closed black-box authority. It enforces governance-backed trust decisions.

Source: On-chain bulletin: Architectural Position; white paper: Governance Model

Effective Service-Node Authorization

Authorization Rule

Authorization is not granted by a credential alone. In OAN, a service node is effectively trusted only when its **current governance state** and its **valid Root-issued authorization** both remain true.



$$\text{Effective Authorization} = \text{Active Governance State} + \text{Valid Root-issued VC}$$

Source: On-chain bulletin: Architectural Position; identity trust layering

Authorization Decision Matrix

Governance State	Root VC	Service Result	Why
Active	Valid	Accept	The node is currently authorized and has an operational credential.
Active	Missing/Invalid	Reject VC-dependent service	Governance decision exists, but the node has not proven Root-issued authorization.
Inactive/Revoked	Valid	Reject new service	A historical VC cannot override latest governance state.
Missing	Valid or missing	Reject	Root cannot infer infrastructure authority without governance state.

Operational Rule

Root maintains a local authorization cache derived from the Trust Indexer and applies it before serving Registrar, Discovery, or third-party VC Issuer nodes.

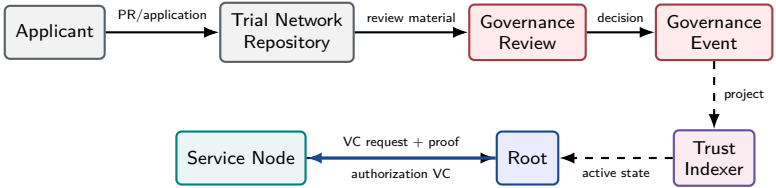
Practical implication: node operators cannot "stay trusted" merely because they once received a credential. Current governance state still wins.

Source: On-chain bulletin design; Root-indexer integration design; genesis authorization tests

Service-Node Admission Lifecycle

Admission Stages

Becoming a trusted OAN service node is a staged admission process, not a single approval click. The figure shows how application material, governance decisions, indexed state, and Root-issued authorization connect into one auditable lifecycle.



Operational Meaning

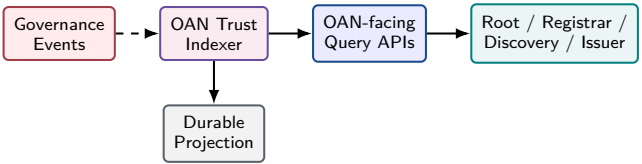
Application review, governance signaling, and Root-issued authorization are separate steps, and each step leaves evidence for audit.

Source: On-chain bulletin: Trial-network Admission Workflow; trust indexer design

Trust Indexer: Decoupling Nodes from Governance Runtime

Why the Indexer Exists

Trust Indexer is the adapter layer between OAN and on-chain governance. It lets OAN consume governance truth consistently across different smart-contract chains, including consortium chains and public chains.



- Reads governance events.
- Reconstructs current subject state.
- Exposes stable OAN trust APIs.
- Adapts heterogeneous chains into one OAN trust view.
- Supports consortium-chain and public-chain governance backends.
- Keeps service nodes away from raw governance runtime complexity.

Design Rule

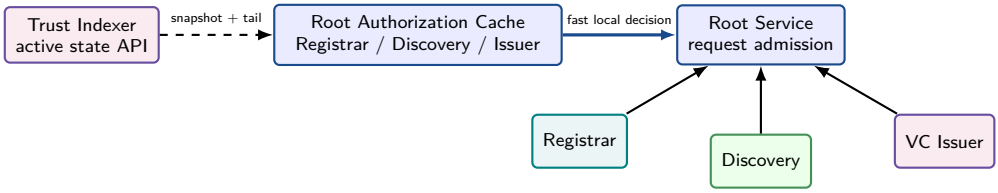
Service nodes consume OAN-oriented trust state from the indexer; the indexer absorbs event pagination, replay, projection, and cursor management.

Source: Trust indexer design: Position, Goals, Normalized Domain Model

Root Runtime Enforcement

Runtime Decision Path

Runtime trust enforcement must be both correct and fast. Instead of consulting raw governance infrastructure on every request, Root continuously absorbs indexed trust state and turns it into local admission decisions that can accept active nodes and cut off revoked ones in time.



- Root does not query raw governance runtime on every service interaction.
- It synchronizes governance state, caches active service-node authority, and stops serving revoked nodes.

Source: Root-indexer integration design; Root service admission logic

Governance State and Authorization VC Boundary

Boundary Clarification

One common source of confusion is the boundary between governance state and credentials. This page shows why a historical authorization VC is not enough on its own if the current governance state no longer supports trust.

Item	What It Proves	What It Does Not Prove
Governance State	The infrastructure subject is active, suspended, or revoked under governance.	It is not a protocol credential and does not prove key control by itself.
Root-issued Authorization VC	Root issued an operational authorization credential to a DID subject.	It is not sufficient if latest governance state is inactive, missing, expired, or revoked.
Subject-Control Proof	Applicant controls the DID key used for the request.	It does not replace governance approval or Root authorization.
Signed Request Envelope	The request body, path, method, audience, nonce, and timestamp were signed.	It does not authorize an unapproved infrastructure node.

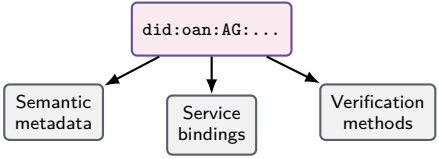
Source: Identity trust layering; on-chain bulletin: Architectural Position; oan-service-security

Why did:oan

Why This Identifier

OAN needs an identifier that stays stable while the surrounding resource details evolve. This page explains why did:oan is the anchor that keeps identity, metadata, service bindings, and proofs moving together without collapsing into endpoint-specific naming.

- Portable identifier for AI-agent-related resources.
- DID document carries verification methods and service bindings.
- Semantic metadata supports discovery by user need.
- Independent of specific invocation protocols.



Specification: did:oan Method Specification.md

Design Intent

did:oan keeps the identifier stable while endpoints, semantic metadata, package versions, and protocol bindings evolve.

Source: DID method spec; did-oan refactor plan; oan-did-oan code

One DID Method, Four Product Forms

One Method, Many Forms

OAN does not invent a separate identity model for every resource category. It uses one DID method to support multiple product forms while keeping registration, trust proof, and discovery semantics aligned.

Product Form	Code	Discovery Meaning
Agent Service	AG	Callable agent service with endpoint and protocol bindings.
Skill	SK	Capability description with artifact references and implementation links.
MCP Server	MC	MCP-compatible server exposing tools, resources, or prompts.
Tool/API	TL	Callable API, connector, workflow, or tool interface.

Shared Model

All four use `did:oan`, DID document metadata, ResourcePackage, Root proof, and Discovery indexing.

Source: Product model: Resource Types; oan-core ResourceType subject-code mapping

Resource Forms: What Is Registered and Found

What Changes by Type

Once the common DID method is established, the next question is what different resource forms actually look like in registration and discovery. This page compares those forms without breaking the shared trust model.

Form	Registered DID Document Carries	Discovery Returns	Consumer Next Step
Agent Service	Endpoint, protocol bindings, semantic profile, keys, VC hints.	Candidate service DID documents and signed evidence.	Verify evidence, then call via supported protocol.
Skill	Semantic description, artifact URI/hash, examples, compatibility metadata.	Candidate Skill DID documents and artifact references.	Fetch artifact from referenced location and verify hash.
MCP Server	MCP endpoint, capabilities summary, resource/tool/prompt metadata.	Candidate MCP server DID documents.	Verify, then connect using MCP.
Tool/API	Endpoint or OpenAPI reference, authentication hints, operation semantics.	Candidate API/tool DID documents.	Verify, then invoke according to API contract.

Unification Point

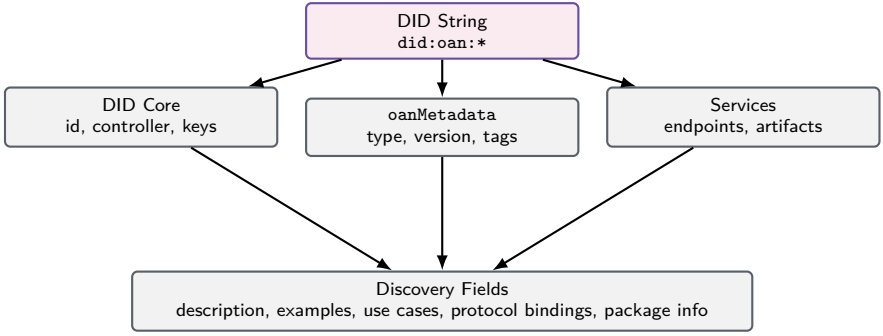
The resource type changes the expected metadata, but registration, Root verification, package proof, and Discovery evidence remain shared.

Source: Product model: four product forms; DID method spec: oanMetadata and services

DID Document as the Discovery Surface

Why DID Documents Matter

After seeing the resource forms, this page zooms in on the common public surface they share. It explains why DID documents are not just identifiers, but also the structured entry point for discovery.



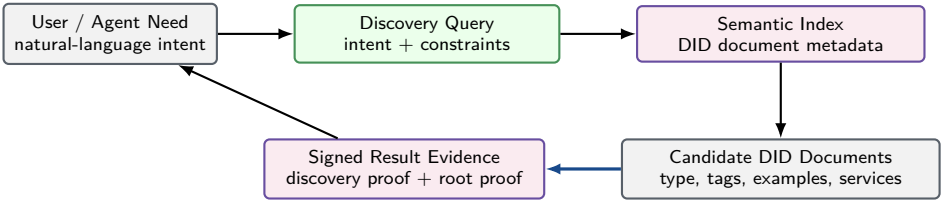
Why this matters

In OAN, Discovery is not indexing an ad hoc registry record. It is indexing the structured public surface of a resource's DID document.

Semantic Discovery Is Built on DID Documents

Where Semantics Lives

Semantic discovery in OAN starts from structured DID-document metadata rather than from ad hoc catalog rows. That is what lets meaning, identity, and trust evidence remain connected through the whole discovery path.



Important Boundary

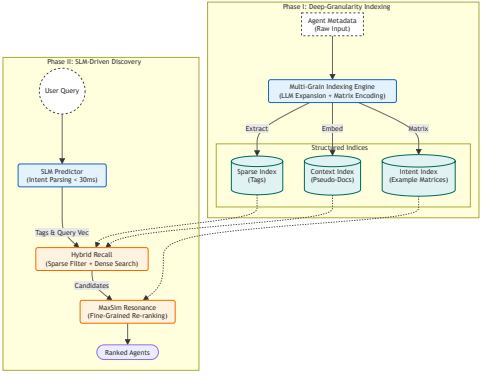
Discovery indexes and returns DID documents with evidence. It does not need to host every external artifact referenced by a resource.

Source: GRAIL paper for semantic discovery direction; product model: discovery boundary

GRAIL: Semantic Discovery Engine at the Discovery Layer

From semantic metadata to semantic retrieval

Once OAN makes semantics part of the DID-document surface, the next question becomes how Discovery should search that surface well at scale. GRAIL is the algorithmic answer at the discovery layer: a purpose-built semantic discovery engine designed for fast, precise agent/resource retrieval under real ecosystem load.



What GRAIL contributes

- **SLM-enhanced prediction:** fast capability-tag prediction instead of heavy online LLM parsing.
- **Tri-dimensional indexing:** tags, context, and intent are modeled separately rather than collapsed into one monolithic vector.
- **MaxSim resonance:** user intent is matched against discrete usage examples to reduce semantic dilution.
- **Real-time target:** the framework is designed for sub-400ms discovery without giving up retrieval quality.

Why it belongs in OAN

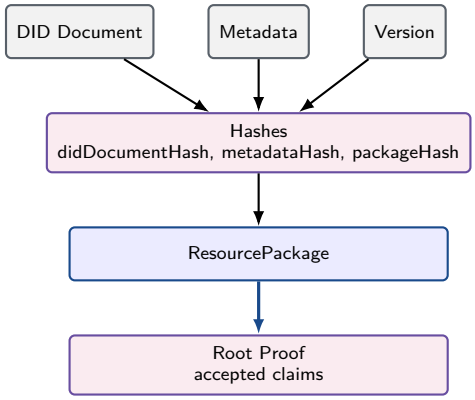
This matters because OAN is not only strong on identity and trust proofs. It also invests deeply in the semantic-search side of discovery, so the Discovery node is backed by both governance-aware trust evidence and a serious retrieval architecture.

Source: GRAIL paper: The Overall Architecture of the GRAIL Framework; semantic discovery engine design

ResourcePackage and Root Proof

Package and Proof Roles

Once a DID document and its metadata are accepted, OAN still needs a publishable object and a portable trust claim. This page explains how the package and the Root proof work together to provide both.



What the package does

The `ResourcePackage` turns a submitted resource into a versioned, hash-bound publication unit that Root can stand behind.

What the Root proof does

The Root proof is the trust bridge for downstream nodes and consumers: it lets them verify that the DID document, metadata, and package version match what Root actually accepted.

Why both are needed

The package structures the publishable object; the proof turns it into verifiable evidence for CDN, Discovery, and end users.

Source: Yellow paper: Protocol Objects; oan-package ResourcePackage and claims

ResourcePackage Binding Rules

What Gets Bound Together

The previous page introduces the package and the proof; this one spells out exactly what they bind together. These bindings are what prevent downstream verification from drifting away from what Root actually accepted.

Binding	Slide-Level Meaning
`resourceDid`	Stable resource subject; not a mutable endpoint or version string.
`resourceType`	Must match DID subject code and DID document `oanMetadata`.
`packageVersion`	Versioned Root-verified release state for the resource.
`didDocumentHash`	Binds the submitted DID document to Root acceptance.
`metadataHash`	Binds semantic and product metadata.
`packageHash`	Binds the package artifact and versioned publication unit.
`rootProof`	Turns Root acceptance into verifiable downstream evidence.

Why These Bindings Matter

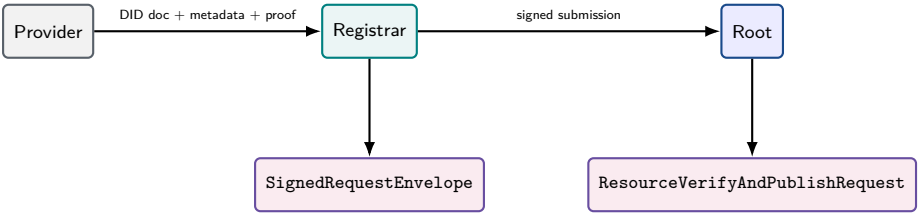
Consumers do not need to trust Discovery blindly; they can verify that returned documents and metadata match Root-accepted package facts.

Source: oan-package ResourcePackage; did-oan maintenance guide hash/proof rules

Resource Registration Sequence

Where Registration Becomes Trust

This is where a provider's resource enters the governed OAN pipeline. The sequence highlights how Registrar forwards a signed submission and how Root checks both authority and content integrity before accepting publication.



Root verifies Registrar authorization, hashes, type consistency, freshness, nonce, and subject-control proof.

Source: System design Trust Flow; oan-protocol ResourceVerifyAndPublishRequest; registrar/root code

Publication and Synchronization

What Happens After Acceptance

Root acceptance is not the end of the lifecycle. The next step is to distribute the verified package across CDN and Discovery without breaking the evidence chain that downstream nodes and consumers depend on.

- CDN distributes Root-approved packages; it is not the trust authority.
- Discovery indexes verified packages and DID document metadata.

Pipeline Boundary

Publication may be asynchronous, but every downstream copy must remain traceable to the same Root proof, package hash, and DID document hash.

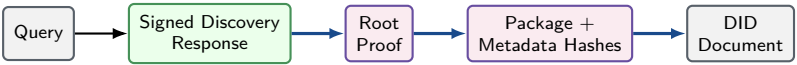
This is the core promise: asynchronous distribution without losing a single trust anchor.

Source: System design route graph; root repository queues; CDN and Discovery service code

Trusted Discovery Chain

The Evidence Path

Discovery trust in OAN is cumulative. A returned candidate is useful only because the response can be traced backward through Root proof, hashes, and DID-document facts to something verifiable.



Discovery with Evidence

Discovery returns candidates that can be traced back to Root-verified resource facts.

In human terms: Discovery is not saying “trust me”; it is saying “here is the evidence chain, verify it yourself.”

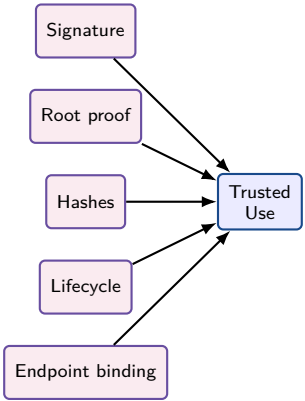
Source: Yellow paper: Discovery Response and verification algorithms; oan-protocol ResourceDiscoveryResponse

What a Consumer Verifies

Verification Checklist

OAN does not ask consumers to trust a search result blindly. It gives them a concrete verification checklist that turns discovery evidence into an informed decision about whether a resource is safe and current enough to use.

- Discovery response signature.
- Root proof and accepted claims.
- DID document hash and metadata hash.
- Package hash and hash algorithm.
- Resource type, lifecycle state, version.
- Endpoint or artifact binding.



Source: Product model: Authentication and Trust Checks; identity trust layering Runtime Verification

Discovery Query Contract

Query Boundary

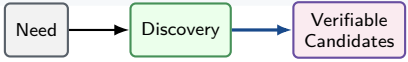
After seeing how resources are published, discovered, and verified, it helps to summarize the actual query boundary. This page frames discovery as a contract between user intent and verifiable candidate evidence.

Query Inputs

- need description and keywords
- optional `resourceType`
- capability tags and custom tags
- protocol binding or endpoint preference
- version or lifecycle constraints

Response Evidence

- candidate `resourceDid`
- type, version, lifecycle state
- capability tags and services
- package info and Root proof reference
- Discovery response proof



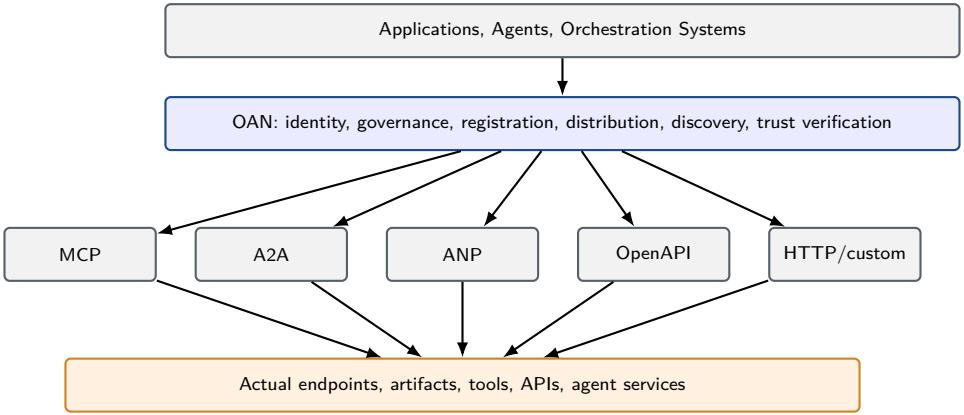
Why this matters for adoption

Developers can keep using familiar protocols after discovery. OAN changes how candidates are identified and trusted, not how every downstream call must be written.

OAN Above Heterogeneous Protocols

Compatibility Boundary

Only after the trust and discovery pipeline is clear does protocol compatibility become easy to place. This page shows that OAN sits above heterogeneous invocation protocols rather than trying to replace them.



Source: White paper: relationship with MCP/A2A/ANP; yellow paper: compatibility boundaries

MCP Server and Tool/API Mapping

Two Resource Profiles

Different resource forms need different metadata surfaces, even when they share the same trust pipeline. This page shows what OAN expects from MCP servers versus tool or API resources so both remain discoverable and verifiable.

MCP Server

- `did:oan` resource type: `mcp_server`.
- Describes endpoint, transport, protocol version.
- Summarizes tools, resources, prompts.

Tool/API

- `did:oan` resource type: `tool_api`.
- Links API schema or OpenAPI description.
- Describes auth, endpoint, examples, checksum.

Source: Product model: MCP Server and Tool/API; refactor plan resource-type profiles

Concrete Example: Registering an MCP Server

A Concrete Walkthrough

The abstract mapping becomes easier to internalize through one concrete case. This example walks through how a familiar MCP server moves through the OAN trust pipeline from registration to verified use.

1. Prepare a DID document that describes the MCP server endpoint, capabilities, verification methods, and metadata.
2. Submit it through a Registrar with signed request evidence.
3. Root verifies the submission and accepts it into the governed publication path.
4. Discovery later returns the MCP server as a signed candidate with evidence.
5. A developer verifies the evidence first, then connects using MCP.

What OAN changes

Without OAN, the developer only sees an MCP endpoint. With OAN, the developer sees a governed identity, Root acceptance, and signed discovery evidence before connecting.

What the developer gains

Less time spent guessing whether the MCP server is real, current, and safe to integrate; more time spent building on a candidate that already carries verifiable trust context.

Human version: OAN helps you check whether the MCP server is a legitimate service or just a random endpoint on the Internet. Source:

Product model: MCP Server flow; Root publication path; discovery evidence path

Agent Service and Skill Mapping

Two Different Roles

Agent services and skills often appear together in product discussions, but they play different roles in the model. One is a callable runtime endpoint; the other is a reusable capability description or artifact reference.

Agent Service

- Callable business Agent.
- Endpoint and protocol bindings.
- May expose A2A, MCP, HTTP, or custom protocols.

Skill

- Capability description.
- Artifact URL, hash, version.
- Implementation links to Agent Service, MCP Server, or Tool/API.

Boundary

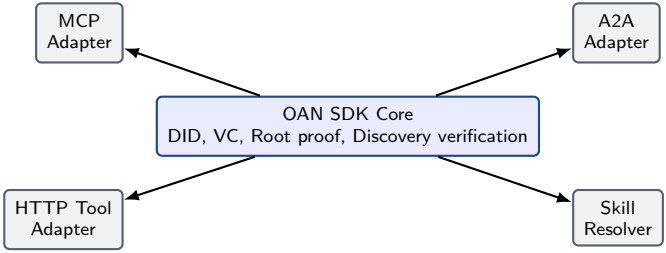
Discovery indexes DID document metadata and references; it does not host Skill files by default.

Source: Product model: Agent Service, Skill, External Artifacts

Adapter and SDK Path

Adoption Path

Adoption gets easier when trust logic is centralized instead of rewritten in every integration. OAN's SDK core provides the shared verification model, while adapters translate it into protocol-specific developer workflows.



Goal

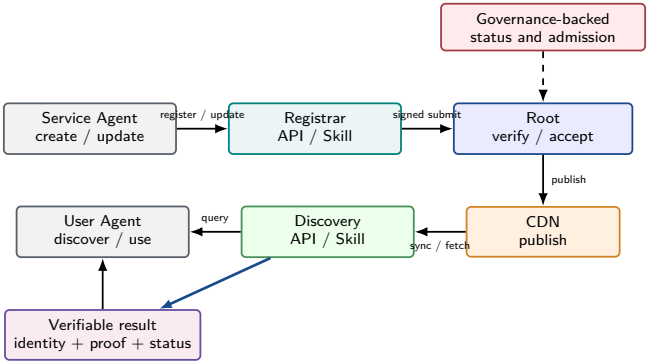
Developers should not hand-roll trust verification; SDKs and adapters should make safe use the default.

Source: Product model: Adapter SDK Direction; yellow paper: SDK Architecture

OAN Is AI-Native Infrastructure

From developer tooling to agent-to-agent infrastructure

The SDK story is only the near-term adoption path. The deeper design point is that OAN exposes machine-consumable trust surfaces, so agents can register, update, discover, verify, and interconnect through OAN directly rather than relying on human operators to drive every step.



What this means

- Agent resources can be registered and updated through Skills or APIs.
- Discovery can return signed candidates that another agent verifies automatically.
- Identity, package, and proof are protocol objects, not only dashboard records.
- Governance-backed status can be consumed by software and folded into automated trust decisions.

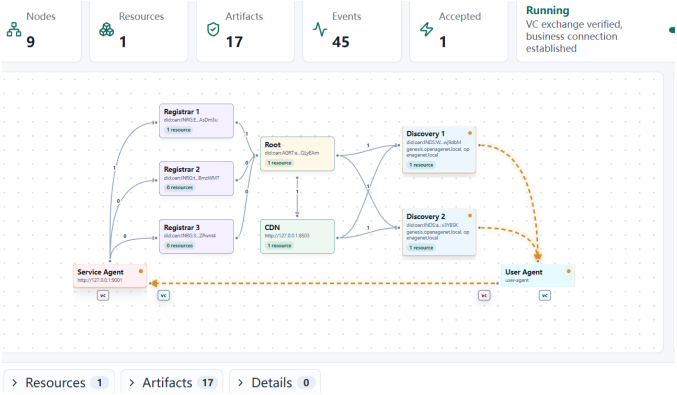
Why this matters

OAN is not just a directory humans browse. It is trust infrastructure that agents can use directly to create, register, discover, verify, and interconnect with one another.

Demo View: One Agent End-to-End

The smallest complete trust loop

One service agent moves through the full OAN path and reaches one user agent, making the complete trust loop easy to read before scale is introduced.

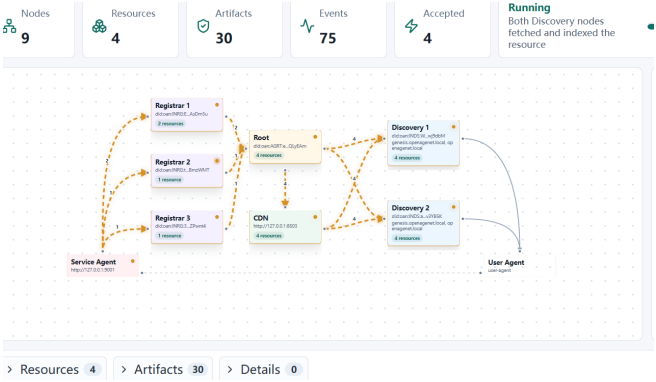


Minimal loop: Registrar submits, Root accepts, CDN publishes, Discovery indexes, and business connection happens only after trust verification.

Demo View: Four Mixed Resources

One topology, multiple resource forms

After the one-agent baseline, this view shows heterogeneous resource publication under the same trust pipeline.



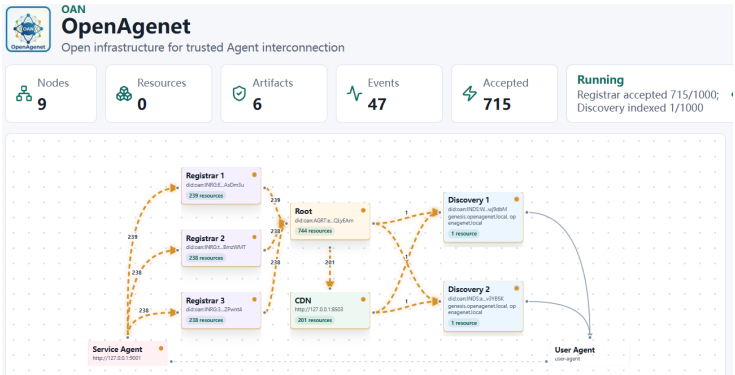
Three Registrars contribute resources, Root aggregates them, CDN republishes them, and both Discovery nodes index the same governed set.

Source: oan-demos scenario screenshot: four-mixed

Demo View: 1000 Mixed Resources

From correctness to operational pressure

This large-scale view turns the benchmark story into something visual: the pipeline is working under volume, and publication plus discovery are advancing under load.



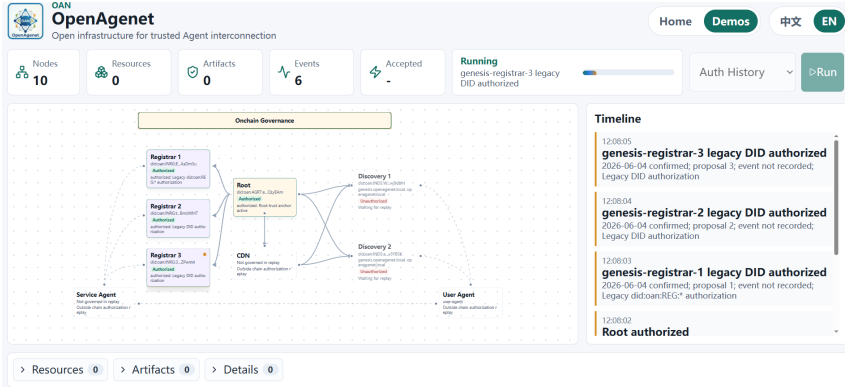
The counters show the key operational fact: Root acceptance leads, publication follows, and Discovery visibility catches up behind the stream.

Source: oan-demos scenario screenshot: 1000-mixed

Demo View: Authorization History

Governance is visible, not hidden

The previous screens emphasize publication and discovery. This one shows the governance side directly as a readable authorization timeline.



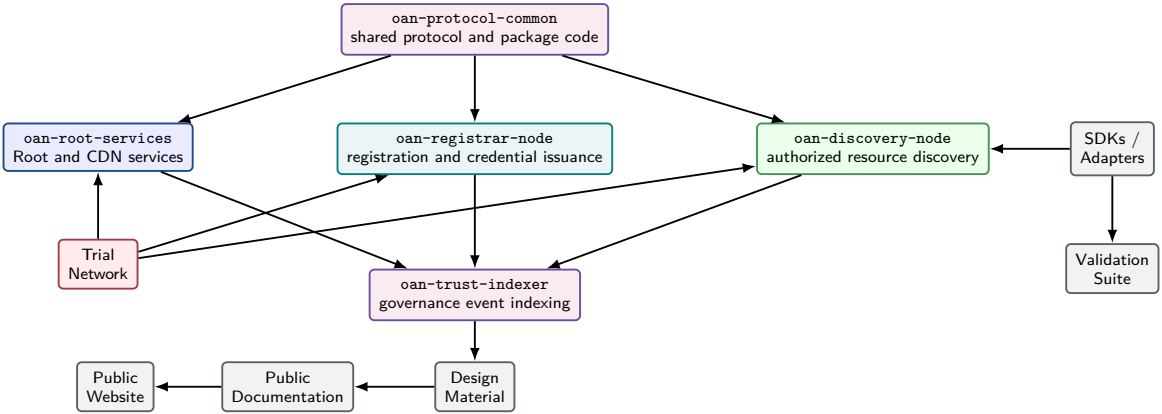
Readers can see not just that nodes are authorized, but when and why they became authorized.

Source: oan-demos scenario screenshot: auth-history

Multi-Repository Engineering Stack

Repository-Level View

After the protocol and lifecycle story, the deck shifts from architecture to engineering organization. This page shows how the implementation is distributed across repositories without fragmenting the trust model itself.



Source: System design: split node repositories and shared protocol libraries

Node Services and Shared Objects

Code Boundary

The implementation boundary matters because OAN is more than one binary. This slide separates operational node roles from shared protocol crates, showing how the system stays modular without fragmenting its trust logic.

Rust Node Services

- Root Node
- Registrar Node
- Discovery Node
- CDN Service

Shared Protocol Crates

- `oan-core`, `oan-did-oan`
- `oan-protocol`, `oan-package`
- `oan-crypto`, `oan-credentials`
- `oan-storage`, `oan-client`

- PostgreSQL-oriented operational state; SQLite remains useful for local development and tests.

Source: System design: Implementation Stack; database abstraction analysis; protocol common crates

Current Formal API Surface

Integration Surface

Once the implementation pieces are visible, the next practical question is where adopters actually integrate. This page summarizes the cleaned-up API surface that exposes the system boundary more clearly than the legacy route layout did.

Role	Route Prefix	Purpose
Root	`/root/...`	Verification, publication, bulletin, authorization, queue operation.
Registrar	`/registrar/...`, `/resources/...`	Resource onboarding, registration submission, capability tree access.
Discovery	`/discovery/...`	Resource synchronization, query, route lookup, signed discovery responses.
CDN	`/cdn/resources/...`, `/cdn/documents/...`, `/cdn/metadata/...`	Distribution of Root-verified packages and document/metadata views.

Current Boundary

The current formal API surface does not retain legacy `/api/v1/...` routes in the mainline service path.

This matters for adopters because the system boundary is becoming cleaner, easier to reason about, and less likely to mislead future integration work.

Documentation and Specification Assets

Documentation Layers

OAN is intentionally not just code. This page shows the internal documentation layers that help the project stay explainable to engineers, reviewers, partners, and future standards-facing audiences.

Asset	Role
White Paper	Vision, value, governance, ecosystem positioning.
Yellow Paper	Technical architecture, protocol objects, lifecycle, security.
DID Method Spec	Public specification surface for `did:oan`.
Detailed Design Docs	System, governance, resource model, indexer, database, website.
Public Docs	Community-facing papers, specs, and selected PDFs.
Slides	Communication material for leadership, partners, developers, and standards.

Message

OAN is being built as code + specifications + governance materials, not only as a service demo.

Source: whitepaper/yellowpaper; did-oan spec; docs/system; oan-public-docs

Public Papers and Standards Output

Public Output

Together this material positions OAN as engineering work, research output, and a candidate standards conversation rather than only a code repository.

White/Yellow Papers

OpenAgenet/OAN White Paper: Open Infrastructure for Trusted Agent Interconnection arXiv:2606.03161

OpenAgenet/OAN Yellow Paper: Technical Architecture for Trust-Governed Resource Identity and Discovery arXiv:2606.03163

Slides: OpenAgenet/OAN Open Governance Infrastructure for Trusted Agent Interconnection github repo

Academic Papers

Trusted Management Model for the Lifecycle of Agent Identity Documents

AONA: A Comprehensive Architecture and Workflow Design for Global Agentic Collaboration

GRAIL: A Deep-Granularity Hybrid Resonance Framework for Real-Time Agent Discovery via SLM-Enhanced Indexing arXiv:2605.02489

DarwinNet: An Evolutionary Network Architecture for Agent-Driven Protocol Synthesis arXiv:2604.01236

IETF Drafts

draft-xu-oan-resource-identity-discovery-00: Trust-Governed Resource Identity and Discovery Architecture for OpenAgenet

draft-xu-agentic-overlay-network-architecture-00: Architecture for Agentic Overlay Networks

draft-xu-efficient-agent-discovery-profile-00: Metadata and Query Profile for Efficient Agent Discovery

Message

OAN is moving as an open-source engineering project, a public research output, and a standards-facing architecture proposal.

Engineering Evidence Behind the Story

Claim-to-Code Link

This page ties the project's external claims to concrete engineering evidence so the narrative stays inspectable.

Claim	Engineering Evidence	Why It Matters
Unified resource identity	`oan-core`, `oan-did-oan`, DID method spec	Identity rules are shared by services, docs, and tests.
Governed service nodes	governance design, Trust Indexer, genesis identities	Infrastructure authorization is observable and testable.
Root-verified publication	`oan-package`, Root service, CDN service	Discovery results can trace back to Root acceptance.
Multi-resource support	validation cases and protocol models for Agent, Skill, MCP, Tool/API	OAN is not limited to one Agent service form.
Developer adoption path	SDKs, adapters, public documentation, and website material	Ecosystem users get practical onboarding material.

Evidence Pattern

Each major claim is backed by both design material and implementation evidence, making the project inspectable rather than merely conceptual.

In other words: this is not just a story about future trust infrastructure; it is already a testable engineering system.

Security and Trust Properties

What OAN Protects

At this point, the key question is not only how OAN works, but what it actually protects. The two columns below separate infrastructure trust from resource trust so readers can see both the guarantees and the deliberate limits.

Infrastructure Trust

- governance state is auditable;
- Root authorization VC is explicit;
- service-node requests are signed;
- revoked nodes are rejected by Root cache.

Resource Trust

- DID document binds keys and services;
- ResourcePackage binds hashes and version;
- Root proof binds accepted claims;
- Discovery response carries verifiable evidence.

Scope

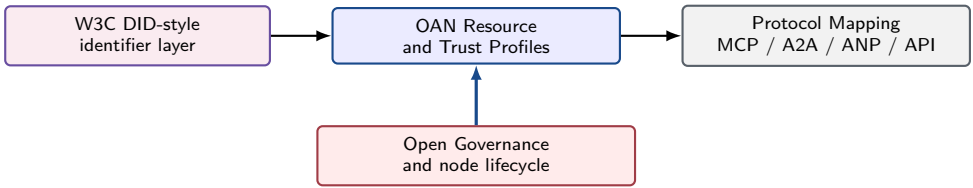
OAN strengthens identity, lifecycle, and discovery trust; it does not replace business-level safety, model evaluation, or endpoint-specific security controls.

Source: Yellow paper security model; identity trust layering; DID method spec security considerations

Standards-Facing Positioning

Standards View

OAN is not just an engineering stack; it is also a candidate architecture that can be described in standards-facing terms. This slide connects its DID layer, trust profiles, governance model, and protocol mappings into that larger picture.



- OAN can be presented as a resource identity and trusted discovery profile for agent ecosystems.
- The project is suitable for open-source cooperation first, and standards-facing discussion as the model stabilizes.

Source: DID method spec; public docs; white/yellow paper standards discussion

Performance Snapshot: What the Current System Already Proves

What the Baseline Already Shows

After the architecture and implementation story, the next question is whether the system already works at meaningful scale. This slide summarizes the current baseline evidence before the deck turns to remaining bottlenecks.

Operational conclusion

- The current implementation already completes **1000-resource** trusted publication and discovery runs with **missing = 0**.
- Single-node mixed-resource pipeline reaches about **57.22 resources/s** at the 1000-resource scale.
- Multi-node mixed-resource pipeline reaches about **47.90 resources/s** at the 1000-resource scale.
- The remaining challenge is no longer correctness; it is throughput and tail latency.

How the four benchmark cases differ

- **Pipeline**: single-node + one resource type.
- **Mixed-resource**: single-node + mixed resource forms.
- **Multi-node pipeline**: multi-node + one resource type.
- **Multi-node mixed**: multi-node + mixed resource forms.

Two comparison axes

- **Topology axis**: single-node vs. multi-node deployment.
- **Resource axis**: one resource form vs. heterogeneous resource mix.

Readable takeaway

The current question is no longer “can OAN work?” It is “how far can OAN scale while preserving governed trust and verifiable discovery?”

Benchmark Reading: Where the System Is Strong and Where It Still Hurts

How to Read the Results

The benchmark snapshot is only useful if it leads to honest interpretation. This page separates what the current data already proves from where the next optimization wave still needs to focus.

Dimension	What the current data already shows	What still needs work
Correctness	Completed baseline runs finish with `missing = 0`.	More operator-scale and long-running evidence is still needed.
Resource coverage	Agent Service, Skill, MCP Server, and Tool/API all run through the same trust pipeline.	Mixed-resource tails are still slower than we want.
Single-node path	Mid-scale throughput is already useful for real engineering validation.	Large-run visibility latency still rises sharply.
Multi-node path	OAN already runs Root + multiple Registrars + multiple Discoveries in one trust model.	Root publication tail and discovery visibility remain the main bottlenecks.

Important message for readers

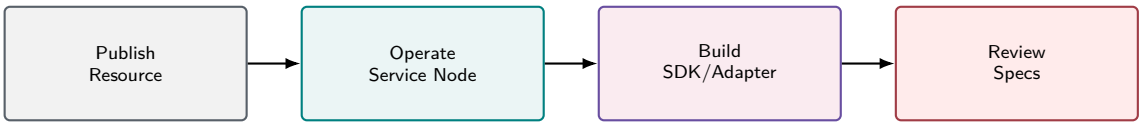
The benchmark tables are engineering evidence, not marketing fantasy. They prove that OAN already works as a real trust pipeline, and they reveal exactly where the next optimization wave should focus.

Source: Performance and scalability analysis; PostgreSQL bottleneck analysis

How to Participate

Ways to Join

Participation in OAN is intentionally broader than code contribution alone. People can strengthen the ecosystem by publishing resources, running governed infrastructure, building adapters, or reviewing the public specifications.



- Publish Agent Service, Skill, MCP Server, or Tool/API resources.
- Apply to operate Registrar, Discovery, or future VC Issuer nodes.
- Contribute adapters, examples, documentation, and conformance tests.

Source: On-chain bulletin admission workflow; product model Registration/Discovery Flow; validation material

Roadmap

What Comes Next

The next stage is about turning OAN into a credible international trust infrastructure path for the Agent Internet: strong in governance, neutral across protocols, operationally mature, and globally legible.

Phase progression: Foundation → Expansion → Global-scale trust infrastructure

Trust Governance Core

Foundation: identity, Root proof, replay-safe authorization
Expansion: conformance, credential lifecycle, governance replay
Scale: federated domains, cross-domain trust handoff

Protocol-Neutral Interoperability

Foundation: MCP, A2A, ANP, and API-facing adapters
Expansion: schema mappings, capability profiles, broader SDKs
Scale: default trust layer for heterogeneous Agent protocols

Operational Maturity

Foundation: performance, observability, reproducible deployment
Expansion: partner nodes, diagnostics, validation suites
Scale: consortium-grade and telecom-grade multi-operator confidence

Ecosystem and Standards Reach

Foundation: papers, demos, public specs
Expansion: pilots across research, industry, and standards communities
Scale: a globally legible trust-governed infrastructure option

Direction of travel: not another monolithic Agent protocol, but a trust and governance substrate for safer interconnection at scale.

Source: Agent interoperability comparison analysis; White paper roadmap; yellow paper Open Technical Work

Call for Collaboration

OAN helps the Agent Internet move from "wild-growth trust" to **verifiable trust infrastructure**.

AI-agent ecosystems need more than protocols. They need rules, evidence, and open trust infrastructure.

Publish	**Operate**	**Build**	**Review**
resources	service nodes	SDKs and adapters	specifications

Next step

Try the code, review the specs, run the demos, and help shape an open trust layer for the Agent ecosystem.

Forwardable one-liner

OAN: the trust layer that helps agent networks move from "I found an endpoint" to "I can verify before I trust".

https://github.com/OpenAgenet

xujinliang@caict.ac.cn; jlxufly@gmail.com

Source: White paper conclusion and stakeholder value; public docs